Customer Behavior Analysis - Complete Project Report

Problem Statement

A leading retail company wants to better understand customers' shopping behavior to improve sales, customer satisfaction and long-term loyalty. The objective is to discover how customer demographics, discounts, reviews, payment methods, seasons and previous purchases influence buying behaviour.

Project Overview

Python was used for data cleaning and feature engineering, MySQL was used to answer business questions through SQL queries, and Power BI was used to build interactive dashboards.

Python Data Cleaning & Preparation

The dataset was explored using head(), info() and describe(). Missing values in Review Rating were filled using the category median. Column names were standardized, Age Group and Purchase Frequency Days were created, Promo Code Used was removed because it duplicated Discount Applied, and the cleaned dataset was uploaded to MySQL.

customer_shopping_behavior

July 21, 2026

[1]:
```
import pandas as pd
import numpy as np
df=pd.read_excel(r"C:\Araib_Projects\customer_shopping_behavior.xlsx")
```

[2]:
```
df.head()
```

[2]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	PayPal	Annually

[3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
```


Figure 1: Dataset Loading and Initial Exploration

```
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   str
 3   Item Purchased          3900 non-null   str
 4   Category                3900 non-null   str
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   str
 7   Size                    3900 non-null   str
 8   Color                   3900 non-null   str
 9   Season                  3900 non-null   str
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   str
 12  Shipping Type           3900 non-null   str
 13  Discount Applied        3900 non-null   str
 14  Promo Code Used         3900 non-null   str
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   str
 17  Frequency of Purchases  3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 548.6 KB
```

[4]:
```
df.describe(include="all")
```

[4]:

	Customer ID	Age	Gender	Item Purchased	Category	\
count	3900.000000	3900.000000	3900	3900	3900	
unique	NaN	NaN	2	25	4	
top	NaN	NaN	Male	Blouse	Clothing	
freq	NaN	NaN	2652	171	1737	
mean	1950.500000	44.068462	NaN	NaN	NaN	
std	1125.977353	15.207589	NaN	NaN	NaN	
min	1.000000	18.000000	NaN	NaN	NaN	
25%	975.750000	31.000000	NaN	NaN	NaN	
50%	1950.500000	44.000000	NaN	NaN	NaN	
75%	2925.250000	57.000000	NaN	NaN	NaN	
max	3900.000000	70.000000	NaN	NaN	NaN	

	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	\
count	3900.000000	3900	3900	3900	3900	3863.000000	
unique	NaN	50	4	25	4	NaN	
top	NaN	Montana	M	Olive	Spring	NaN	
freq	NaN	96	1755	177	999	NaN	
mean	59.764359	NaN	NaN	NaN	NaN	3.750065	
std	23.685392	NaN	NaN	NaN	NaN	0.716983	
min	20.000000	NaN	NaN	NaN	NaN	2.500000	

Figure 2: Dataset Information and Summary Statistics

25%	39.000000	NaN	NaN	NaN	NaN	3.100000
50%	60.000000	NaN	NaN	NaN	NaN	3.800000
75%	81.000000	NaN	NaN	NaN	NaN	4.400000
max	100.000000	NaN	NaN	NaN	NaN	5.000000

	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	\
count	3900	3900	3900	3900	
unique	2	6	2	2	
top	No	Free Shipping	No	No	
freq	2847	675	2223	2223	
mean	NaN	NaN	NaN	NaN	
std	NaN	NaN	NaN	NaN	
min	NaN	NaN	NaN	NaN	
25%	NaN	NaN	NaN	NaN	
50%	NaN	NaN	NaN	NaN	
75%	NaN	NaN	NaN	NaN	
max	NaN	NaN	NaN	NaN	

	Previous Purchases	Payment Method	Frequency of Purchases
count	3900.000000	3900	3900
unique	NaN	6	7
top	NaN	PayPal	Every 3 Months
freq	NaN	677	584
mean	25.351538	NaN	NaN
std	14.447125	NaN	NaN
min	1.000000	NaN	NaN
25%	13.000000	NaN	NaN
50%	25.000000	NaN	NaN
75%	38.000000	NaN	NaN
max	50.000000	NaN	NaN

[5]:
```
df.isnull().sum()
```

[5]:
```
Customer ID                0
Age                        0
Gender                     0
Item Purchased             0
Category                   0
Purchase Amount (USD)      0
Location                   0
Size                       0
Color                      0
Season                     0
Review Rating             37
Subscription Status        0
Shipping Type              0
Discount Applied           0
```

Figure 3: Missing Value Analysis

```
Promo Code Used           0
Previous Purchases        0
Payment Method            0
Frequency of Purchases    0
dtype: int64
```

```
[6]: df["Review Rating"] = df.groupby("Category")["Review Rating"].transform(lambda
     x: x.fillna(x.median()))
```

```
[7]: df.isnull().sum()
```

```
[7]: Customer ID              0
     Age                      0
     Gender                   0
     Item Purchased           0
     Category                 0
     Purchase Amount (USD)    0
     Location                 0
     Size                     0
     Color                    0
     Season                   0
     Review Rating            0
     Subscription Status      0
     Shipping Type            0
     Discount Applied         0
     Promo Code Used          0
     Previous Purchases       0
     Payment Method           0
     Frequency of Purchases   0
     dtype: int64
```

```
[8]: df.columns = df.columns.str.lower()
     df.columns = df.columns.str.replace(' ','_')
     df = df.rename(columns={'purchase_amount_(usd)':'purchase_amount'})
```

```
[9]: df.columns
```

```
[9]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
            'purchase_amount', 'location', 'size', 'color', 'season',
            'review_rating', 'subscription_status', 'shipping_type',
            'discount_applied', 'promo_code_used', 'previous_purchases',
            'payment_method', 'frequency_of_purchases'],
           dtype='str')
```

```
[ ]: # create a column age_group
     df["age_group"] = df["age"].transform(lambda x:
```


Figure 4: Handling Missing Values and Standardizing Columns

```
                                        "Young Adult" if x<=30 else 'Adult' if␣
↪x<=45 else 'Middle-aged'
                                        if x<=60 else 'Senior')
```

[11]:
```
df[["age", "age_group"]].head(10)
```

[11]:

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Adult
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

[12]:
```
# create a column purchase_frequency_days
frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}
df["purchase_frequency_days"] = df["frequency_of_purchases"].
↪map(frequency_mapping)
```

[13]:
```
df[["purchase_frequency_days", "frequency_of_purchases"]].head(100)
```

[13]:

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
..	…	…
95	30	Monthly
96	90	Every 3 Months
97	365	Annually
98	365	Annually
99	90	Quarterly

[100 rows x 2 columns]

Figure 5: Creating Age Group and Purchase Frequency Days

```
[14]: df[["discount_applied", "promo_code_used"]].head(10)

[14]:   discount_applied promo_code_used
        0              Yes             Yes
        1              Yes             Yes
        2              Yes             Yes
        3              Yes             Yes
        4              Yes             Yes
        5              Yes             Yes
        6              Yes             Yes
        7              Yes             Yes
        8              Yes             Yes
        9              Yes             Yes

[15]: (df["discount_applied"] == df["promo_code_used"]).all()

[15]: np.True_

[16]: df = df.drop("promo_code_used", axis=1)

[17]: df.columns

[17]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
              'purchase_amount', 'location', 'size', 'color', 'season',
              'review_rating', 'subscription_status', 'shipping_type',
              'discount_applied', 'previous_purchases', 'payment_method',
              'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
             dtype='str')

! pip install psycopg2-binary sqlalchemy

! pip install pymysql

[ ]: import pandas as pd
     from sqlalchemy import create_engine

     # my sql connection detail
     username = "root"
     password = "malik2000#"
     host = "127.0.0.1"
     port = "3306"
     database = "customer_behavior"

     # create mysql connection
     engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/
      ↪{database}")
     print("my sql connected successfully")
```


Figure 6: Removing Redundant Column and MySQL Connection

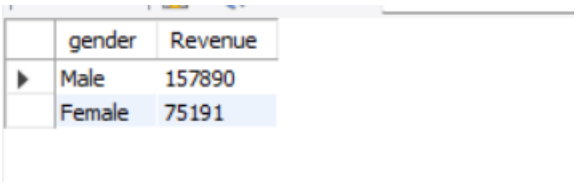
```
# upload dataframe to my sql
df.to_sql(name="customer_data", con=engine, if_exists="replace", index=False)
print("data uploaded successfully")

[ ]:
```

Figure 7: Uploading Cleaned Dataset to MySQL

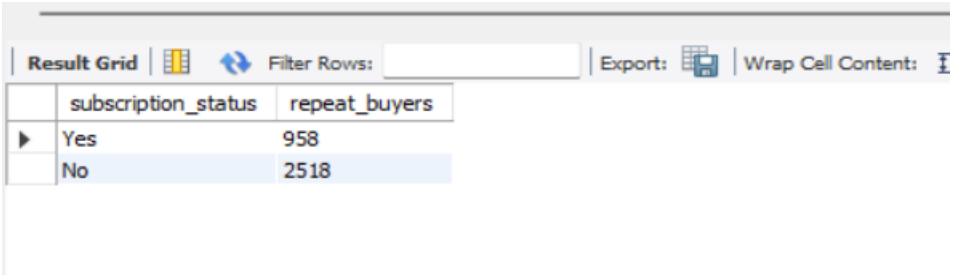
SQL Business Analysis

Business questions were answered using SQL. Each query helped identify important patterns in customer behaviour.



gender	Revenue
Male	157890
Female	75191

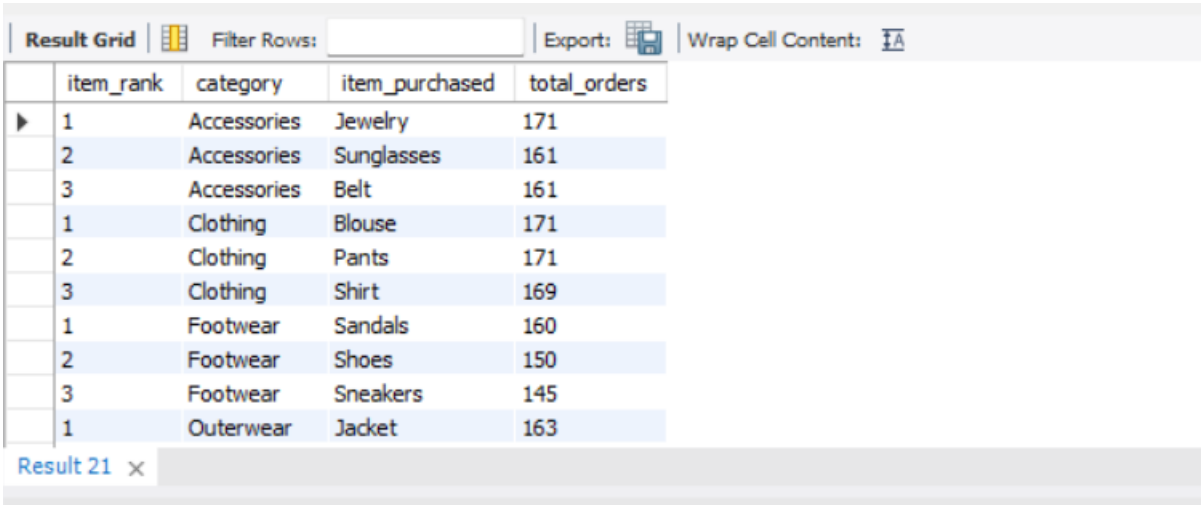
Revenue by Gender



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

subscription_status	repeat_buyers
Yes	958
No	2518

Repeat Buyer by Subscription Status



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_rank	category	item_purchased	total_orders
1	Accessories	Jewelry	171
2	Accessories	Sunglasses	161
3	Accessories	Belt	161
1	Clothing	Blouse	171
2	Clothing	Pants	171
3	Clothing	Shirt	169
1	Footwear	Sandals	160
2	Footwear	Shoes	150
3	Footwear	Sneakers	145
1	Outerwear	Jacket	163

Result 21

Top 3 Best Selling Product in Each Category

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_segment	number of customer
loyal	3116
returning	701
new	83

Customer segment Analysis

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	discount_rate
Hat	50.00
Sneakers	49.66
Coat	49.07
Sweater	48.17
Pants	47.37

Top 5 Product with Highest Discount Rate

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

subscription_status	total_customer	average_spend	total_revenue
Yes	1053	59.4919	62645
No	2847	59.8651	170436

Average Spending and Total Revenue by Subscription Status

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

shipping_type	avg(purchase_amount)
Express	60.4752
Standard	58.4602

Average Spending by Shipping Type

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	average_product_rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.8
Skirt	3.78

Top 5 Product by Average Review Rating

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	purchase_amount
2	64
3	73
4	90
7	85
9	97
12	68
13	72
16	81
20	90
22	62

customer_data 15 ×

Customer who used Discount and Spent above Average

Result Grid | Filter Rows: |

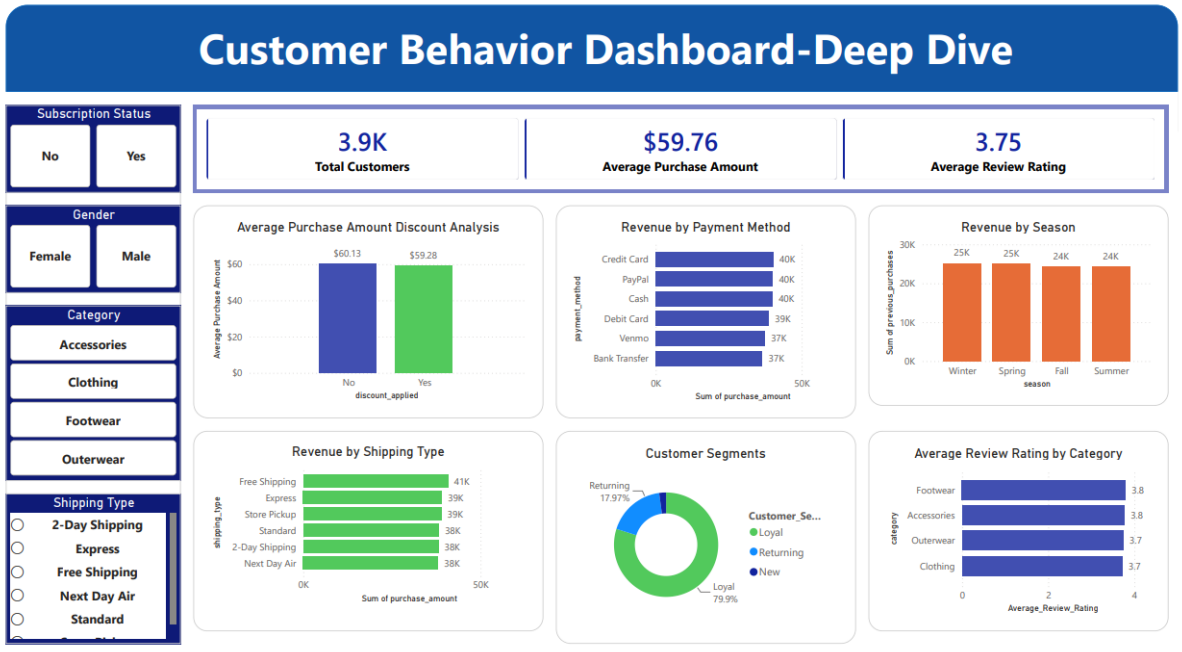
age_group	total_revenue
Middle-aged	67711
Adult	64927
Young Adult	57279
Senior	43164

Revenue by Age Group

Power BI Dashboard

Dashboard Page 1

This page provides KPIs, category analysis, customer distribution and age group analysis.



Dashboard Page 2

This page focuses on payment methods, shipping types, customer segments, review ratings and seasonal performance.

Business Recommendations

1.Focus more on middle-aged customers because they generated the highest revenue.
2. Continue promoting clothing products because they performed well.
3. Encourage loyal customers with simple reward offers.
4. Promote highly rated products on the homepage.
5. Plan seasonal offers before demand increases.
6. Review the dashboard regularly to track customer behaviour.

Conclusion

This project demonstrates a complete Data Analytics workflow using Python, MySQL and Power BI. The findings can help management improve sales and customer decision-making